Homework Seven : More on Trees and Sorting

Question One

> Implement maketree, setleft, and setright for right in-threaded binary trees using the sequential array representation

We can build this using similar pieces from last time:

```
struct threaded_tree_node
    value
    int left
    int right
    bool right_threaded
end

function node * build_threaded_tree(value)
    threaded_tree_node new_node = new();
    new_node->value = value;
    new_node->right = NULL;
    new_node->left = NULL;
    new_node->right_threaded = false
    return new_node
endfunction: build_threaded_tree

function set_left(parent, left_child)
    parent->left = left_child;
endfunction: set_left

function set_right(parent, right_child, threaded)
    parent->right = right_child
    parent->right_threaded = true
endfunction: set_right
```

Question Two

> Implement inorder traversal for the right in-thread tree in the previous problem

```
function leftmost(node)
    while (node->left != NULL) begin
        node = node->left
    end
    return node
endfunction: leftmost

function inorder_traversal(root)
    // get the leftmost node from the root
```

```
    current = leftmost(root)

    if(current->right_threaded) begin
        current = current->right
    end
    else begin
        current = leftmost(current->right)
    end

endfunction: inorder_traversal
```

Question Three

> Let's sort using a method not discussed in class. Suppose you have n data values in in array A. Declare an array called Count. Look at the value in A[i]. Count the number of items in A that are smaller than the value in A[i]. Assign that result to Count[i]. Declare an output array Output. Assign Output[count[i]]= A[i]. Think about what the size of Output needs to be. Is it n or something else? Write a method to sort based on this strategy

So output will only be N, because the number of items smaller than a given item will be unique for each item in A. Thus, the value of count will be unique, meaning the value of Count[i] will be unique for each i in n.

```
    function arr non_class_sort(A)
        n = A.size();
        count, output = new(n)

        for(int i = 0; i < n; i++) begin
            for(int j = 0; j < n; j++) begin
                if(A[i] > A[j]) begin
                    count[i]++;
                end
            end
            // Just do this here because this is the only time we'll modify
count[i]
            out[count[i]] = a[i]
        end
        return out
    endfunction: non_class_sort
```

Question Four

> Analyze the cost of the sort you wrote in the previous problem. What is the impact of random, ordered, or reverse ordered data?

Time complexity: nested for loop of n, so O(n^2) Space Complexity: O(2n) -> O(n)

Impact of data ordering: Because this compares against every known value no matter the order of the data, ordered, reverse ordered, and random data will always have O(n) space and O(n^2) time complexity, so there

is no impact on ordering

Question Five

> How many comparisons are necessary to find the largest and smallest of a set of n distinct elements? Do not assume the answer must involve sorting. It could but does not need to do so. Try to be as efficient as you can.

The basic algorithm I know of from prior experience is the last-known, which has you set a min and max value to the value of the first element of a set of n elements. Then, you traverse each element and compare it against the min and max. If the value is less than the min, it replaces the min. If it is greater than the max, it replaces the max. There's a moticum of efficiency that can be gained by doing either the min or max only if the max or min fails, reducing total comparisons. However, in the worst case, which is defined by the minimum and the maximum being the last traveled to nodes, respectively, you'll have to do n total travels, with two comparisons per travel except for the last you go to. Thus, it'll be a total of 2(n-1) - 1 total comparisons.

Since we don't care about the values other than min and max, we can make this o(n) time, which is the most efficient manner. In a best case, we'd only have to make only n - 1 comparisons.